

Assignment 08 – Creating Applications

Introduction

This application enables the user to add employees by first and last name, track employee ratings, and rating dates.

When the application is opened, employee data is read in from the “EmployeeRatings.json” file and the user is presented with a menu of options that includes:

- Show current employee rating data.
- Enter new employee rating data.
- Save data to a file.
- Exit the program.

Upon exit, the user is prompted to save the data entered to the “EmployeeRatings.json” file.

Data entry notes:

Employee Names – employee first and last names may be single or compound names and, with the exception of the apostrophe symbol, may not contain numbers or symbols. For example, “Mary Sue O’Connor”.

Employee Rating Dates – dates must be in the format YYYY-MM-DD. For example, “2026-02-25”.

Employee Ratings – ratings range from 1 – 5. For example, “4”.

Data Classes

The application contains the Person and the Employee data classes detailed in the “data_classes.py” script.

The **Person** class is used to store a first and last name. Names may be single or compound names and, with the exception of the apostrophe symbol, may not contain numbers or symbols. For example, “Mary Sue O’Connor.” Upon initialization, the person class defaults the first and last name to empty strings.

The following methods are available to the Person class.

class Person:

```
def __init__(self, first_name: str = "", last_name: str = ""): # Initializes a Person object
def first_name(self): # Returns first name
def first_name(self, value: str): # Sets first name
def last_name(self): # Returns last name
def last_name(self, value: str): # Sets last name
def __str__(self): # Returns first and last name as comma separated values
```

The **Employee** class extents the Person class by adding employee rating and employee rating date fields. Ratings range from 1 – 5 and employee rating dates are in the “YYYY-MM-DD” format. Upon initialization, the employee rating is set to 3 and the rating date is set to “1900-01-01”.

The following methods are available to the Employee class.

class Employee (Person):

```

# Initializes an Employee object
def __init__(self, first_name: str = "", last_name: str = "", review_date: str = "1900-01-01", review_rating: int = 3):
def review_date (self): # Returns review date
def review_date (self, value: str): # Sets review date
def review_rating (self): # Returns rating
def review_rating (self, value: int): # Sets rating
def __str__(self): # Returns first name, last name, review date, and rating as comma separated values.

```

Processing Classes

The application contains the FileProcessor and IO classes.

The **FileProcessor** class is located in the “processing_classes.py” file and is used to read data to and from a JSON file.

The following methods are available to the FileProcessor class.

class FileProcessor:

```

def read_employee_data_from_file(file_name: str, employee_data: list, employee_type: object):
# Read data from a JSON file. The data is expected to be in the following JSON format.
[
{
"FirstName": "Vic",
"LastName": "Vu",
"ReviewDate": "2025-02-12",
"ReviewRating": 5
}
]

def write_employee_data_to_file(file_name: str, employee_data: list):
# Write data from a JSON file. The data is saved in the following JSON format.
[
{
"FirstName": "Vic",
"LastName": "Vu",
"ReviewDate": "2025-02-12",
"ReviewRating": 5
}
]

```

The **IO** class is located in the “presentation_classes.py” file and contains methods to interact with the user.

The following methods are available to the IO class.

class IO:

```

def output_error_messages(message: str, error: Exception = None): # Generate error message
def output_menu(menu: str): # Display menu
def input_menu_choice(): # Get user menu choice
def output_employee_data(employee_data: list): # Display employee data
def input_employee_data(employee_data: list, employee_type: object): # Get new employee information from user

```

Main.

The file “main.py” is used to execute the application.